

# KNN Classifier

AUTHORS:

KRYSTIAN RUSZCZAK – MAIN DEVELOPER

PAWEŁ MICHALCEWICZ – TESTER / REPO MAINTAINER

REVIEWED BY: JAKUB ŁUDZIK, FILIP ŻUREK

# Goal of the project

---

The goal of this project is to implement the k-Nearest Neighbors (KNN) classifier from scratch and compare its performance with the official Scikit-learn implementation.

The evaluation is performed on several standard benchmark datasets:

- Iris,
- Wine,
- Breast Cancer,
- Digits,

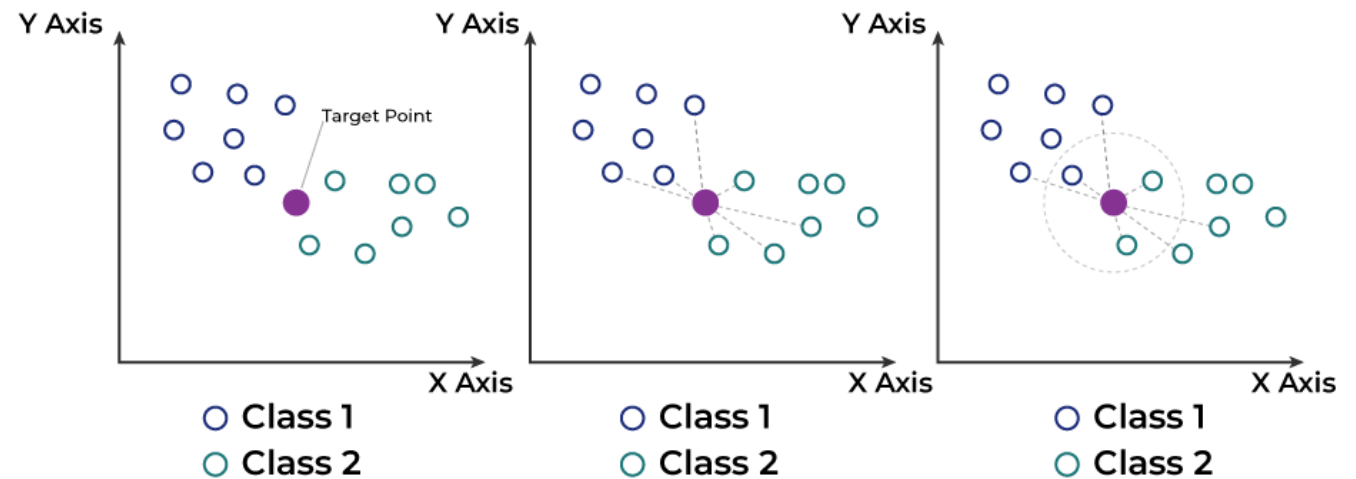
using different values of the K parameter. This project allows for validating the correctness, reliability, and efficiency of the custom KNN implementation.

# Algorithm Overview

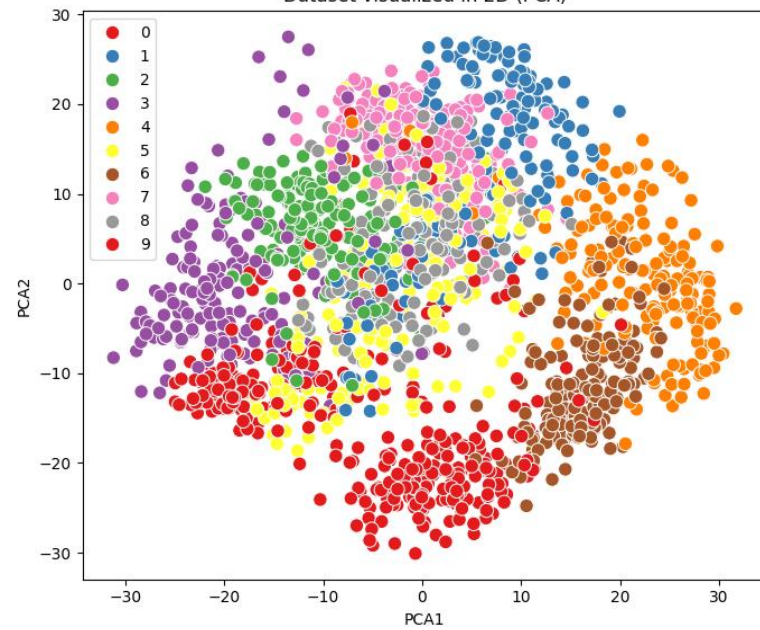
The k-nearest neighbors algorithm (k-NN) is a supervised learning method.

It does not build an explicit model during training. Instead it stores the training data and makes predictions only when new input samples are provided.

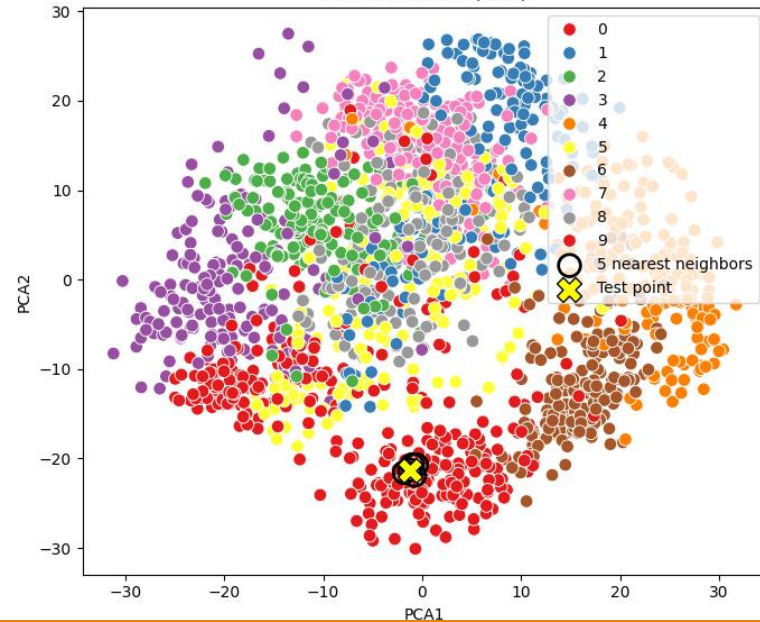
To classify new data, the algorithm looks at the k closest samples in the training set and assigns the class that appears most frequently among them.



Dataset visualized in 2D (PCA)



KNN Illustration (K=5)



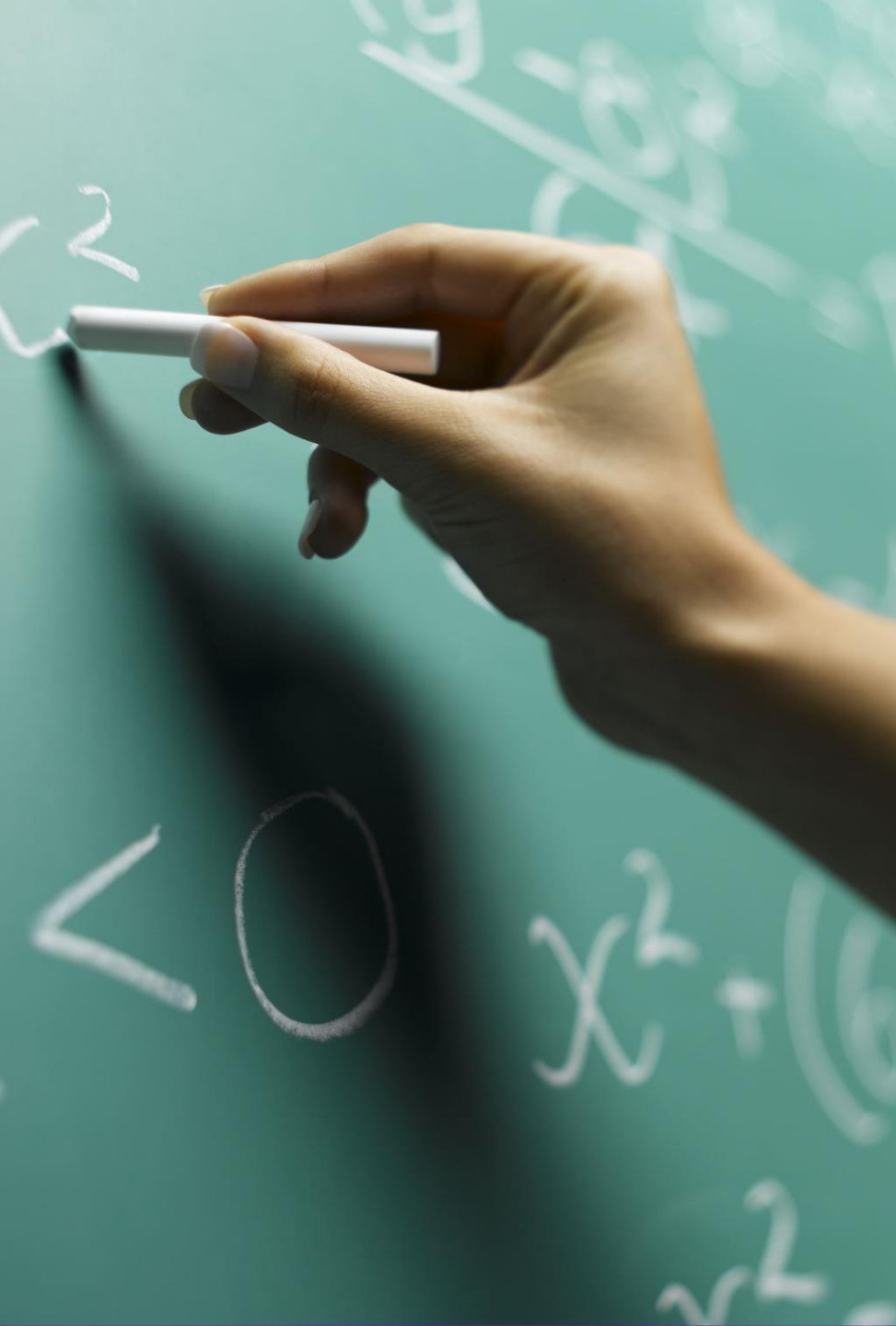
# Euclidean distance

The closeness of two data points is measured using a distance metric - Euclidean distance.

For two dimensions in the Euclidean plane, let point p have Cartesian coordinates (p1, p2) and let point q have coordinates (q1, q2). Then the distance between p and q is given by:

$$d(p, q) = \sqrt{\sum_{i=1}^N (p_i - q_i)^2}$$

For N dimensions.



# Implementation

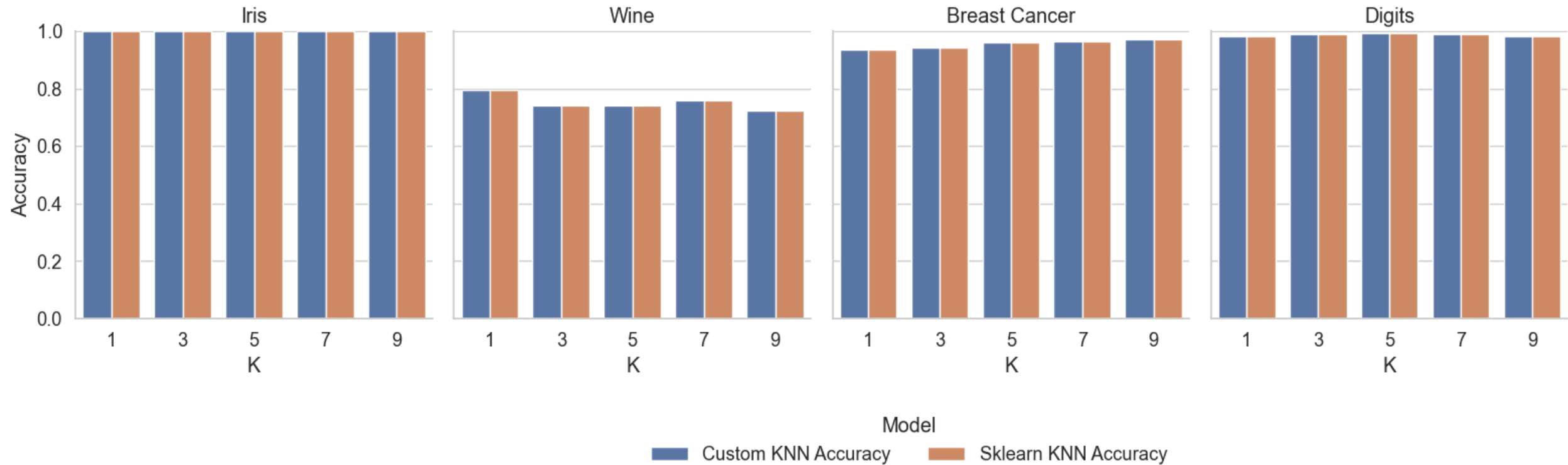
---

The class KNNClassifier contains following methods:

- fit,
- \_euclidean\_distance,
- \_predict\_single,
- predict,
- score.

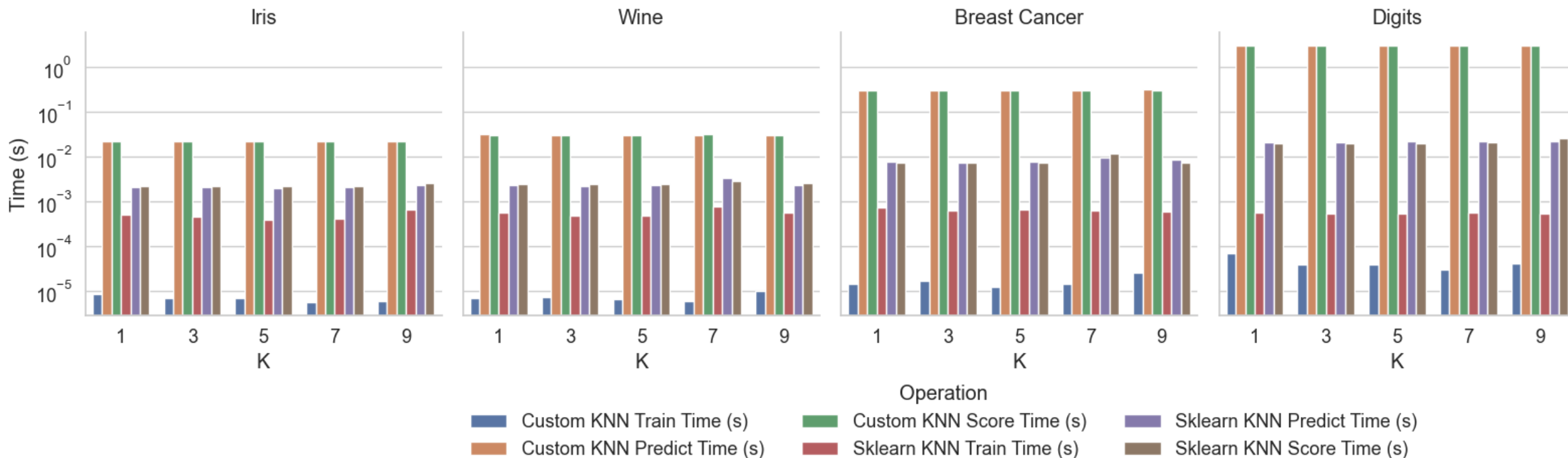
# Comparison between two implementations

4 datasets were selected for the experiment: Iris, Wine, Breast Cancer, Digits. For each dataset, models were tested using five values of k: 1, 3, 5, 7 and 9.



# Performance Comparison

**Prediction and Scoring Time** The most significant discrepancy appears in prediction speed: For small datasets (Iris, Wine), the custom KNN is slower than scikit-learn by roughly one order of magnitude. For medium datasets (Breast Cancer), the difference grows to 40 times slower. For the largest dataset (Digits), the difference becomes dramatic: Custom KNN prediction time 3.0 seconds, scikit-learn prediction time around 0.02 seconds.



# Performance Analysis: Why is Scikit-learn Faster?

---

## Algorithmic Approach Differences:

### Custom KNN:

Uses a "Brute-force" approach. It calculates the distance to *every* point in the training set for each query.

Our implementation relies on pure Python/NumPy computations.

### Scikit-learn:

Utilizes spatial indexing structures (**KD-Tree** or **Ball Tree**), allowing it to efficiently ignore distant points without calculation.

Scikit-learn uses optimized **Cython/C** code for critical operations, eliminating the Python interpreter overhead.





# Tests

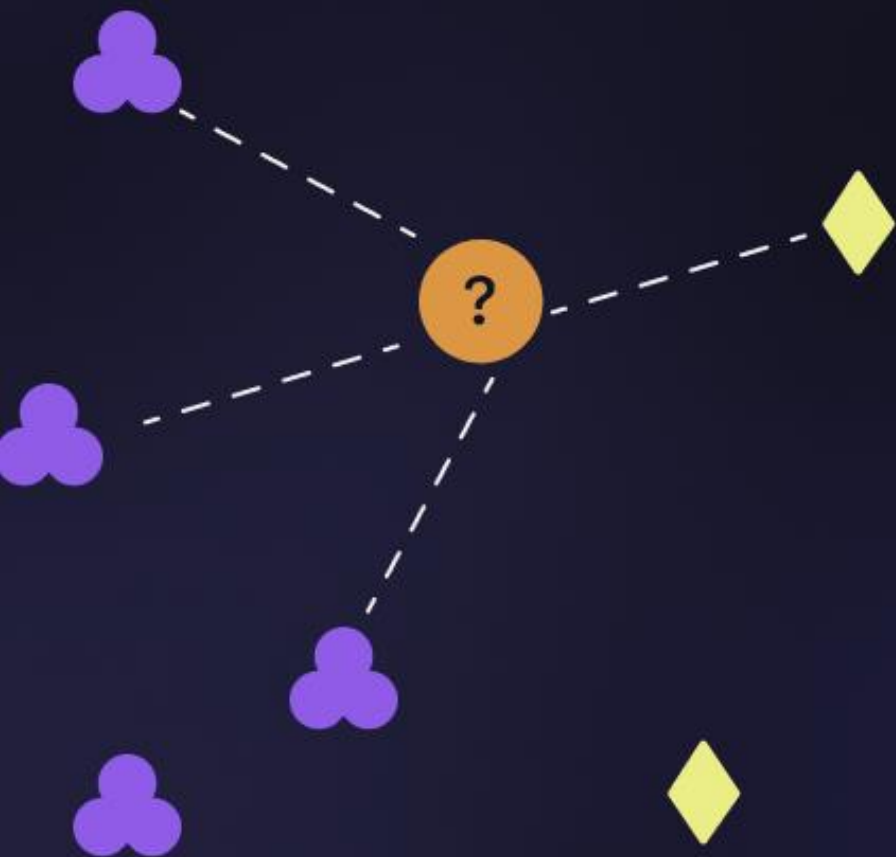
---

The custom KNN model was checked using several small unit tests and one larger integration test. The unit tests made sure that the model starts with the right settings (`test_initialization()`), correctly stores the training data (`test_fit_storage()`), calculates distances properly (`test_euclidean_math()`), and gives the expected predictions on simple examples (`test_prediction_logic()`).

The final integration test compared the whole model to scikit-learn's KNN on the Iris dataset (`test_compare_with_sklearn()`). Both models returned same results, which confirms that the implementation works correctly and behaves just like the standard KNN classifier.



TEST



# Summary

## Goal Achieved:

- Successfully implemented a functional KNN classifier consistent with the algorithm's logic.
- Prediction accuracy is identical to the reference Scikit-learn implementation.

## • Key Observations:

- **Performance:** A simple implementation is sufficient for small datasets (e.g., Iris) but does not scale well for larger data (e.g., Digits).
- **Optimization:** Building algorithms from scratch highlights that Data Science is not just about mathematics, but also about engineering efficient, optimized code.

## • Future Work:

- Implementation of spatial indexing structures (e.g., KD-Trees) to improve speed.
- Adding support for other distance metrics (Manhattan, Cosine, Minkowski).

---

Thank You For Your Attention

